
SYNOPSIS – Phase 3

Water Planning for Urban Household

Project Id: AIML PBL9

Member 1 (Team Leader):

Name: Aryan Arora

Class Roll No.: 35

University Roll No.: 2028666

Section: K

Group: G1

Member 2:

Name: Deepak Rana

Class Roll No.: 10

University Roll No.: 2028410

Section: K

Group: G1

Member 3:

Name: Aayush Bhala

Class Roll No.: 26

University Roll No.: 2028207

Section: K

Group: G1

TABLE OF CONTENTS

1. Problem Description3
 1.1 Dataset Description.....3
 1.2 Flow Diagram4
2. Modules for Project Implementation5
 2.1 Modules Used5
 2.2 Platform Used6
3. Machine Learning Models Used7
4. Evaluation Metrics Used8
5. Results and Visualizations.....9
6. Conclusion and Future Scope13
7. References14

1. Problem Description

Water is one of the most important resource that every household needs on daily basis. In urban areas the demand for water keeps increasing as population grows and more people shift to cities. The main problem is that municipal bodies and water supply authorities dont always know how much water each household is consuming, which makes it difficult to plan supply properly.

This project tries to solve that problem by using machine learning to predict the daily water usage of a household. We take different features like how many people live in the house, what type of building it is, income level, number of appliances, garden area, temperature etc. and train a model that can predict how many litres of water that household will use per day.

The main goals of this project are –

- Find out which factors effect the water usage the most
- Clean and prepare the raw data so models can be trained on it
- Train and compare 3 different regression models
- Check the accuracy of each model using different evaluation methods
- Save the best model using pickle so it can be used later for predictions

1.1 Dataset Description

The dataset we used is called urban_water_demand.csv. It contains data about different urban households and their daily water consumption. The last column Daily_Liters_Used is our target variable which we are trying to predict.

Feature Name	Type	What it Means
Household_Size	Float	How many people are living in the house
Seasonal_Index	Float	Shows how season effects water demand (like summer uses more)
Garden_Area	Float	Area of garden or outdoor space in square metres
Avg_Temperature_C	Float	Average temperature at that location in Celsius
Num_Appliances	Integer	Number of water using appliances like washing machine, dishwasher etc.
Income_Level	Categorical	Whether household income is Low, Medium or High
Building_Type	Categorical	Type of house like Apartment, Villa or Bungalow
Daily_Liters_Used	Float	Target column – actual water used per day in litres

Missing values – Some columns had null values in them. For Household_Size, Garden_Area and Num_Appliances we filled missing values with the median, and for Seasonal_Index and Avg_Temperature_C we used mean value. After doing this there were no missing values left in the dataset.

For Income_Level and Building_Type which are text type columns, we used LabelEncoder from sklearn to convert them into numbers because ML models cant work directly on string data.

1.2 Flow Diagram

Below is the step by step flow of how the project works –

Load Dataset (urban_water_demand.csv)
Check shape, columns, data types
Handle Missing Values
Encode Categorical Columns
EDA – Graphs and Visualizations
Detect Outliers using IQR
Split data – 80% Train / 20% Test
Apply StandardScaler and MinMaxScaler
Train 3 Models (LR, Lasso, Ridge)
Evaluate using R2, MSE, MAE, Accuracy, F1
Plot Confusion Matrix for all models
Save best model to model.pkl
Load model and predict for new input

2. Modules for Project Implementation

2.1 Modules Used

Below are all the python libraries and modules that we have used in this project and what they are used for –

Module	Purpose
pandas	Used to read the CSV file and do all data operations like checking null values, groupby, creating dataframes etc. Most of the data cleaning part is done using pandas.
numpy	Used for mathematical operations, mainly for calculating quantile values when finding outliers using IQR method.
matplotlib.pyplot	Used for making all the basic graphs like histogram, bar chart, scatter plot, boxplot etc.
seaborn	Used for the correlation heatmap and the error distribution plot with KDE line.
pickle	Used to save the trained model and scaler together into model.pkl and then load it back for making predictions.
warnings	Used to suppress unnecessary warning messages that come during training.
StandardScaler	Scales features so that mean becomes 0 and standard deviation becomes 1. Used in the final model pipeline.
MinMaxScaler	Scales features between 0 and 1. Used here to compare how scaling looks against StandardScaler.
LabelEncoder	Converts text columns Income_Level and Building_Type into numbers so the model can understand them.
train_test_split	Splits the data into training (80%) and testing (20%) parts.
LinearRegression	Basic linear regression model. Used as the main model and also saved in.pkl file.
Lasso	Linear regression with L1 regularization. Can reduce some feature coefficients to zero which helps in feature selection.
Ridge	Linear regression with L2 regularization. Good when features are correlated with each other.
r2_score	Tells how much variance in the target is explained by the model. Closer to 1 means better.
mean_squared_error	Average of squared differences between actual and predicted values.
mean_absolute_error	Average of absolute differences. Less sensitive to outliers than MSE.
accuracy_score	After bucketing predictions to Low/Medium/High, gives percentage of correct predictions.
f1_score	Weighted average of precision and recall across all three demand categories.
confusion_matrix / ConfusionMatrixDisplay	Generates and plots the confusion matrix for all 3 models.

2.2 Platform Used

- Jupyter Notebook – The whole code is written in a .ipynb file. We used Jupyter because it lets you run code cell by cell and see outputs and graphs right away.

- Python 3 – The programming language used for this project. Python has many ready made libraries for data science and machine learning.
- Google Colab / Anaconda – The notebook can run on both platforms. We mostly used Jupyter through Anaconda locally.

3. Machine Learning Models Used

We have used three regression models in this project. All three are from the linear_model module of sklearn. The reason we chose these three is because they are simple, fast and easy to compare with each other.

Model	Type	Description
Linear Regression	Regression	Most basic model. Fits a straight line through the data by minimizing squared errors. Used as base model and also saved in pkl file.
Lasso Regression	Regression with L1	Adds a penalty on absolute value of coefficients. Because of this some coefficients become exactly zero which means Lasso removes features that are not useful.
Ridge Regression	Regression with L2	Adds a penalty on square of coefficients. Doesn't remove any feature but reduces effect of correlated features, helps in reducing overfitting.

All three models are trained on x_train and y_train. Predictions are made on x_test and stored separately. We noticed all three give similar results on this dataset which shows the data has mostly a linear relationship with the target variable.

Feature Scaling

We applied two different scalers to see how they affect the feature distribution. Comparison was shown as histogram for Household_Size column –

Scaler	Output Range	Formula
StandardScaler	Mean = 0, Std = 1	$z = (x - \text{mean}) / \text{std}$
MinMaxScaler	0 to 1	$x' = (x - \text{min}) / (\text{max} - \text{min})$

StandardScaler is used in the final pipeline saved to pkl because regularized models work better when all features are on the same scale.

4. Evaluation Metrics Used

We used both regression metrics and classification metrics to evaluate the models. Classification metrics are used after we bucket the predictions into Low, Medium and High categories.

Metric	Formula	What it tells us
R2 Score	$1 - (\text{res} / \text{tot})$	How well the model explains the data. Value of 0.85 means model explains 85% of variation in water usage.
MSE	$\Sigma(y - \hat{y})^2 / n$	Average of squared errors. Larger errors are penalized more. Lower is better.
MAE	$\Sigma y - \hat{y} / n$	Average of absolute errors. Easy to understand as it is in same unit as target (litres).
Accuracy Score	Correct / Total (after bucketing)	After converting to Low/Medium/High, what percentage was correctly classified.
F1 Score	$2 \times (P \times R) / (P + R)$	Combines precision and recall. Useful when categories are slightly imbalanced.

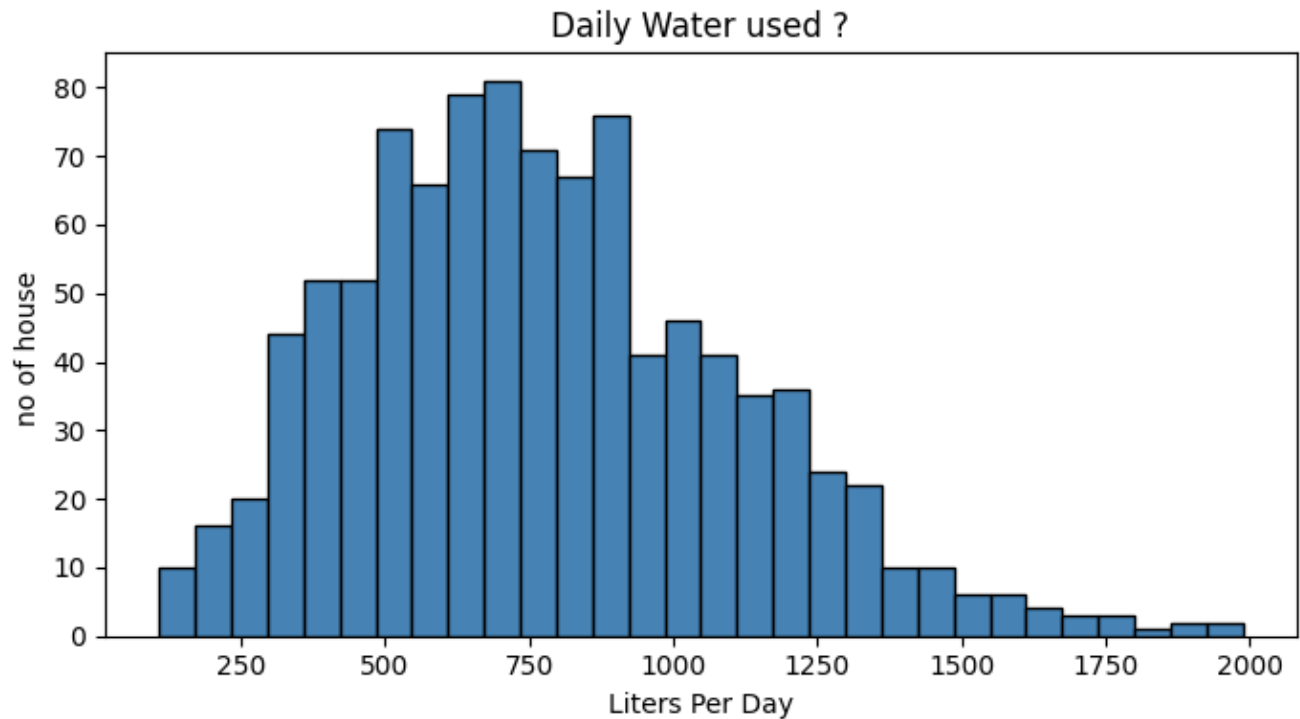
For converting predictions to categories we used a function called `categorize()` –

Category	Condition	Meaning
Low	Less than 500 litres/day	Small family or efficient usage
Medium	500 to 999 litres/day	Normal urban household consumption
High	1000 litres/day or more	High consumption, may need water conservation measures

5. Results and Visualizations

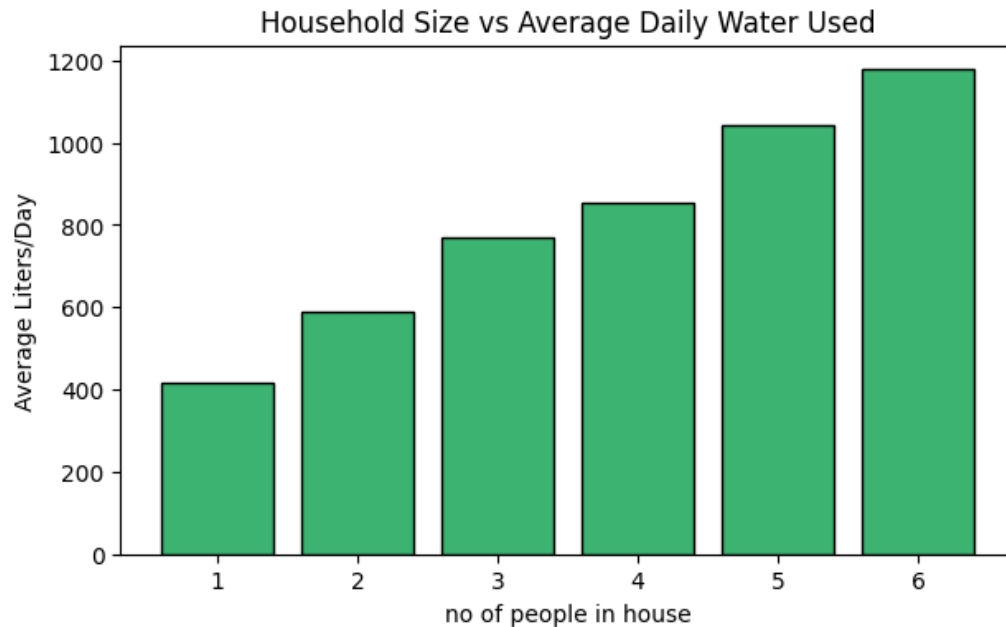
The notebook generates 7 different plots. Each plot helps in understanding the data or model performance better.

Fig 1 – Histogram of Daily Water Used



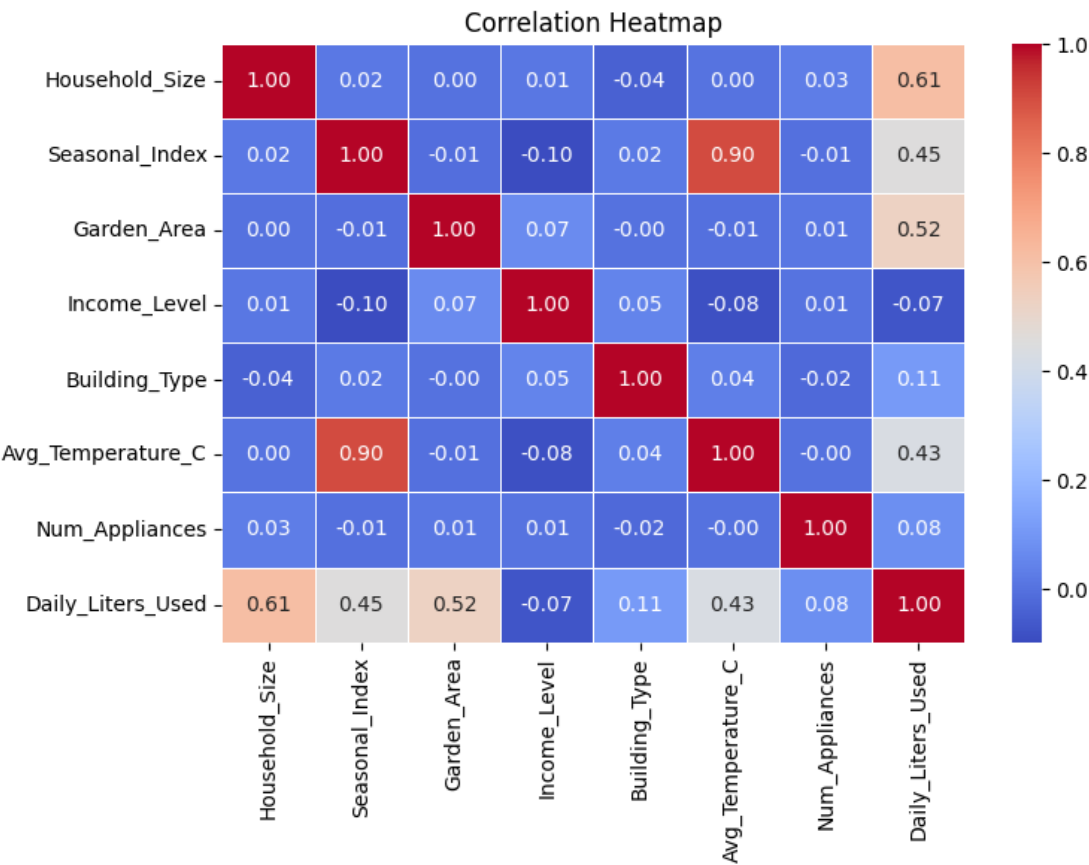
A histogram of Daily_Liters_Used column with 30 bins. Shows how water consumption is distributed across all households. Helps us see if the data is normally distributed or skewed.

Fig 2 – Household Size vs Average Daily Water Used



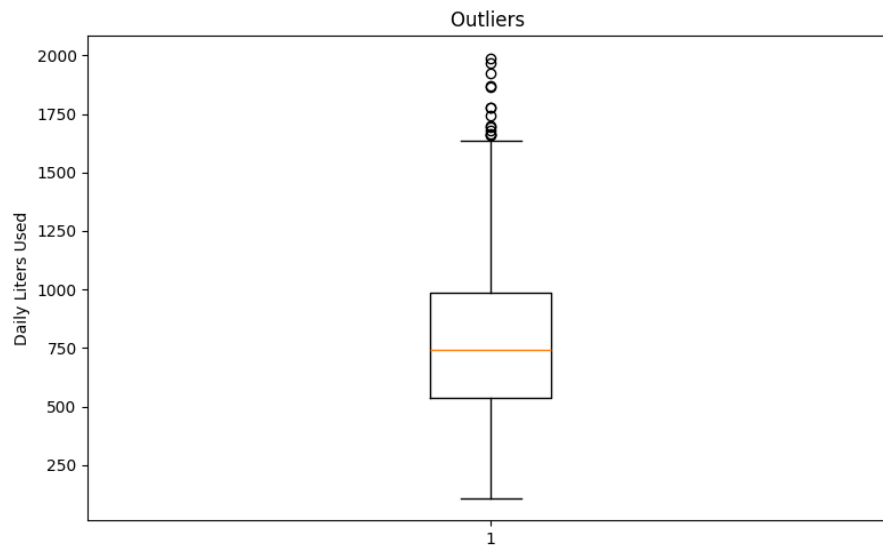
Bar chart that groups households by size and shows average water consumption for each group. As household size increases, water usage also increases which confirms Household_Size is an important feature.

Fig 3 – Correlation Heatmap



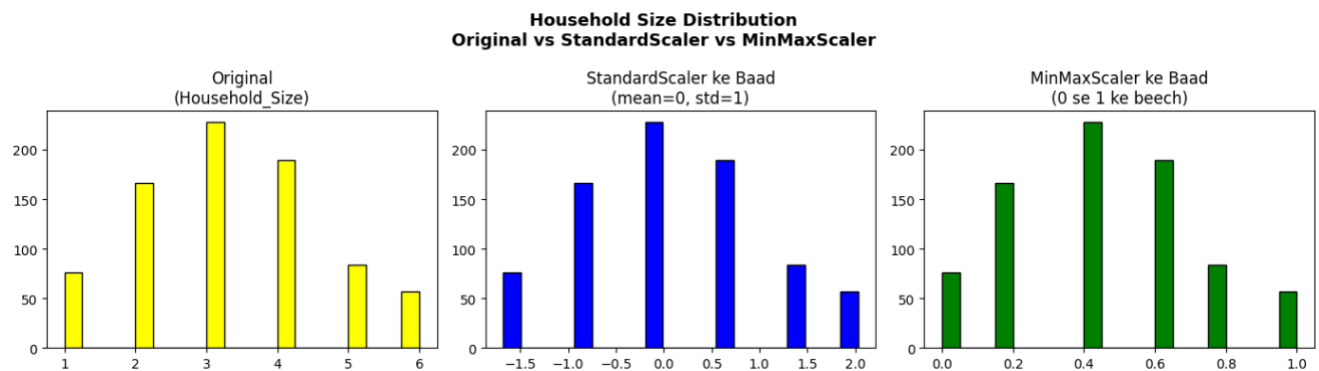
Heatmap showing pairwise correlation between all features. Features with high correlation to Daily_Liters_Used are more useful for prediction. High correlation between two features indicates multicollinearity which is one reason we tested Ridge regression.

Fig 4 – Boxplot with Outlier Detection



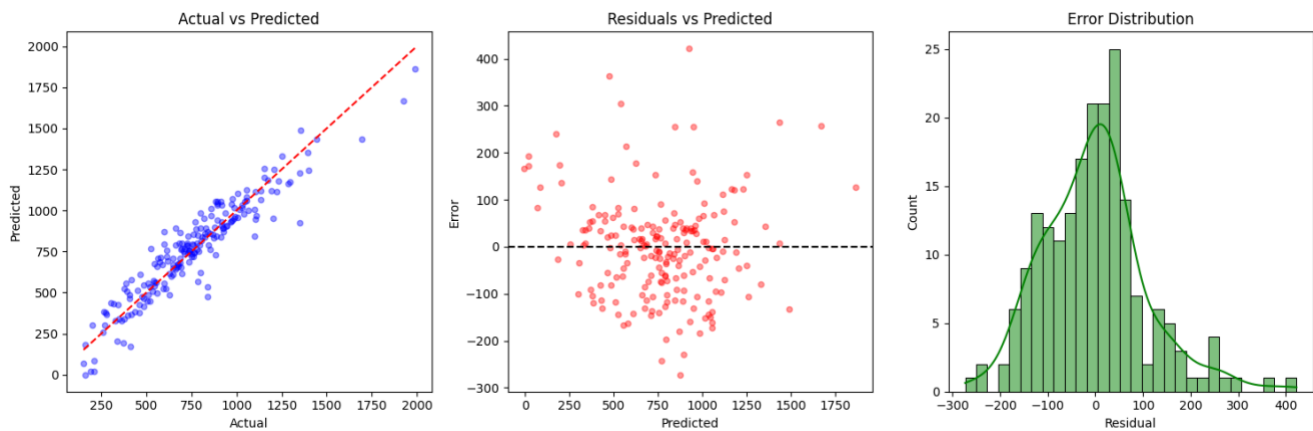
Boxplot of `Daily_Liters_Used` along with IQR based outlier count shown in the title. Helps us understand if there are extreme values in the data that could affect model training.

Fig 5 – Scaling Comparison



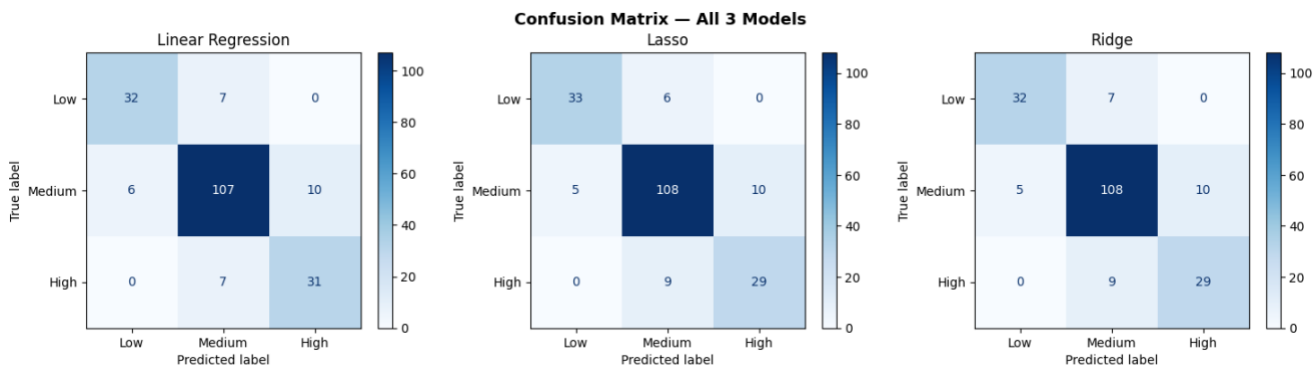
Three histograms placed side by side for `Household_Size` column – original data, after `StandardScaler`, and after `MinMaxScaler`. The shape of data remains same, only the scale on x-axis changes.

Fig 6 – Actual vs Predicted, Residuals and Error Distribution



A 3 panel plot for Linear Regression. First plot shows actual vs predicted values with a diagonal reference line. Second shows residuals vs predicted to check for patterns in errors. Third shows distribution of residuals which should be close to normal and centered around zero.

Fig 7 – Confusion Matrix for all 3 Models



Three 3x3 confusion label matrices plotted side by side for Linear Regression, Lasso and Ridge. Shows how many samples from each category were correctly classified and how many got confused with other categories.

6. Conclusion and Future Scope

Conclusion

In this project we built a machine learning pipeline to predict daily water consumption of urban households. The dataset had 8 features and we did proper preprocessing like handling missing values, encoding and scaling before training the models.

We trained three models – Linear Regression, Lasso and Ridge and all three gave similar results which shows that water demand has a mostly linear relationship with the given features. The model is saved using pickle so that it can be used later without retraining. Overall the project achieves its goal of predicting household water demand and can be useful for urban water planning.

Future Scope

- If we collect data over time then time series models like LSTM can be used to capture seasonal trends better.
- The saved model.pkl can be deployed as a web API using Flask or FastAPI so that users can get predictions in real time.
- More features like age group of residents, type of job or proximity to water supply can be added to improve accuracy.

7. References

Books

- Aurélien Géron – Hands-On Machine Learning with Scikit-Learn, Keras and TensorFlow
- Yashavant Kanetkar – Let Us Python

Online Resources

- scikit-learn Documentation – <https://scikit-learn.org/stable/>
- pandas Documentation – <https://pandas.pydata.org/docs/>
- Towards Data Science – <https://towardsdatascience.com>
- TutorialsPoint – <https://www.tutorialspoint.com>